

Efficient Data Storage Systems

University of Minho

Guide 7

Data replication

2026

The main goal of this guide is to design and implement a replication mechanism in a distributed file system using FUSE. The system should store file data on remote servers while keeping file metadata locally at the client.

1 Setup

Use the VM and FUSE environment from previous guides, along with the remote file system implemented in Guide 6. The client and storage servers will run on the same VM, communicating via the `localhost` interface. Ensure all necessary directories (e.g., `/backend`, ...) are created. Reuse and adapt the code from the previous guide to complete the exercises described below.

2 Exercises

For the following exercises, assume a flat file system namespace (*i.e.*, all files reside in the root directory).

1. Extend the implementation of the remote file system from Guide 6 to consider **two storage nodes** instead of one. Whenever a file is written, it should be stored in both storage nodes. Ensure that storage nodes have different folders for storing client data. Try out your program (and measure the performance) by writing and reading different files.
2. Update your implementation to support a *passive replication scheme*.
 - Client writes must be submitted to a single storage server (*leader*).
 - Replication occurs in background between servers (*leader* and *followers*).
 - The client must block until the file is persisted in *all* storage servers (*synchronous*).
 - Client reads can be submitted to either storage server.

Hint: To determine the storage server for a file, generate a hash based on the file name and use its parity (even/odd) to select a server.

3. Update the *passive replication scheme* implementation to support *asynchronous* writes. Specifically, the client should only block until the file is persisted in the *leader*. Compare the performance of both implementations and simulate different crash scenarios where servers can fail.

3 Extras

1. Enhance your system to support *active replication*, where:
 - Client writes are submitted to all storage servers.
 - The client must block until the file is persisted in *all* storage servers.
 - Consider having multiple clients writing/reading to/from the same files.
 - Compare the *passive* and *active replication schemes* in terms of performance and durability guarantees.
2. Extend your implementation to support more than two storage servers.
3. Implement a *data recovery mechanism* that restores lost files if a storage server fails.

Learning outcomes

Implement a data replication scheme in a distributed file system with FUSE. Understand the design implications of having synchronous and asynchronous replication, as well as passive and active replication.